Computer Graphics in OpenGL
INM376 / IN3005

Module Leader – Dr Eddie Edwards

Lecture 03: Viewing, VBOs, and splines

Recap from last time

- Primitives
  - Points
  - Lines
  - Triangles
- Matrices
  - Theory
  - Transformations: translation, rotation, scaling
  - Modeling using GLM matrices and a matrix stack

2x2 Matrix Inversion

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

$$AA^{-1} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

$$A^{-1} = \frac{1}{ad - bc}\begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

(ad-bc) is the *determinant*

Example from last week?

$$A = \begin{pmatrix} 9 & -2 \\ -3 & -4 \end{pmatrix}$$

$$A^{-1} = \frac{1}{9 * -4 - (-2 * -3)}\begin{pmatrix} -4 & 2 \\ 3 & 9 \end{pmatrix}$$

$$= \frac{1}{-42}\begin{pmatrix} -4 & 2 \\ 3 & 9 \end{pmatrix}$$

$$= \begin{pmatrix} \frac{2}{21} & \frac{-1}{21} \\ \frac{-1}{14} & \frac{-3}{14} \end{pmatrix}$$

2x2 Rotation Matrix Inversion?

$$R = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

$$Det(A) = \cos\theta * \cos\theta + \sin\theta * \sin\theta = 1$$

$$RR^{-1} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

$$R^{-1} = \begin{pmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{pmatrix}$$

$$= \begin{pmatrix} \cos(-\theta) & -\sin(-\theta) \\ \sin(-\theta) & \cos(-\theta) \end{pmatrix} = R^T$$

So the inverse of a 2D rotation matrix is just a rotation by $-\theta$

The inverse of a 2D rotation matrix is it's transpose

Rotations satisfy $RR^T = I$ (orthogonality)

3x3 Matrix Determinant?

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix}$$

$$det(A) = \begin{vmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{vmatrix} =$$

$$a_{11} \begin{vmatrix} a_{22} & a_{23} \\ a_{32} & a_{33} \end{vmatrix} - a_{12} \begin{vmatrix} a_{21} & a_{23} \\ a_{31} & a_{33} \end{vmatrix} + a_{13} \begin{vmatrix} a_{21} & a_{22} \\ a_{31} & a_{32} \end{vmatrix}$$

3x3 Matrix inverse (adjugate method)

- Take the transpose:

$$A^T = \begin{pmatrix} a_{11} & a_{21} & a_{31} \\ a_{12} & a_{22} & a_{32} \\ a_{13} & a_{23} & a_{33} \end{pmatrix}$$

$$Adj(A) = \begin{pmatrix} \begin{vmatrix} a_{22} & a_{32} \\ a_{23} & a_{33} \end{vmatrix} & -\begin{vmatrix} a_{12} & a_{32} \\ a_{13} & a_{33} \end{vmatrix} & \begin{vmatrix} a_{12} & a_{22} \\ a_{13} & a_{23} \end{vmatrix} \\ -\begin{vmatrix} a_{21} & a_{31} \\ a_{23} & a_{33} \end{vmatrix} & \begin{vmatrix} a_{11} & a_{31} \\ a_{13} & a_{33} \end{vmatrix} & -\begin{vmatrix} a_{11} & a_{21} \\ a_{13} & a_{23} \end{vmatrix} \\ \begin{vmatrix} a_{21} & a_{31} \\ a_{22} & a_{32} \end{vmatrix} & -\begin{vmatrix} a_{11} & a_{31} \\ a_{12} & a_{32} \end{vmatrix} & \begin{vmatrix} a_{11} & a_{21} \\ a_{12} & a_{22} \end{vmatrix} \end{pmatrix}$$

$$A^{-1} = \frac{1}{\det(A)} Adj(A)$$

3x3 Rotation Matrix Properties

- In 3D as in 2D, rotations satisfy the orthogonality condition
$$RR^T = I$$
- The columns or rows of R form an orthonormal basis
  - Each column is a vector of length 1
  - The three columns are orthogonal
  - The columns define a set of axes that span
  - Can be seen as a rotation from one coordinate system to another (source and target space)
- The inverse, $R^{-1} = R^T$, represents the inverse transformation i.e. a rotation from the target coordinate space back to the source space

3D rotation

- Rotation in 3D is about an axis. Rotations about the three coordinate axes are

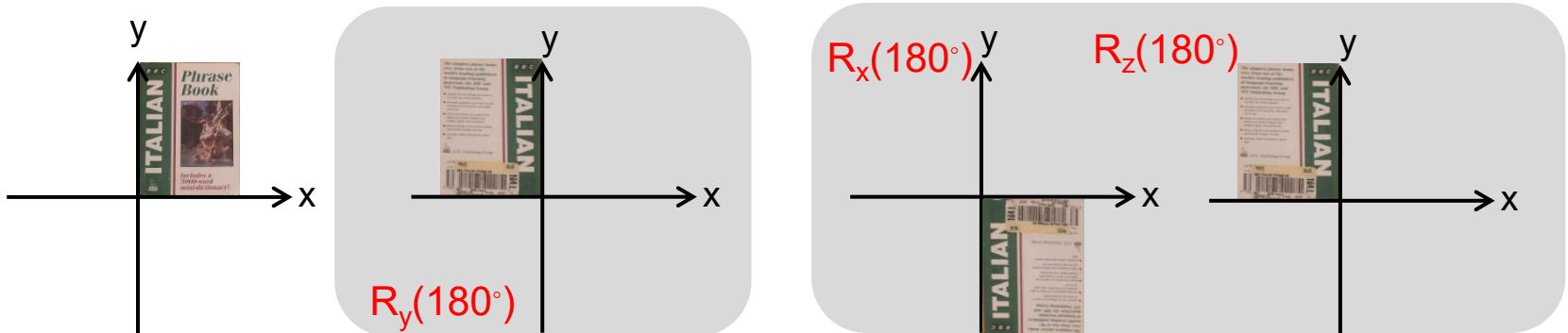
$$R_x(\psi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\psi & -\sin\psi \\ 0 & \sin\psi & \cos\psi \end{bmatrix} \qquad R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix} \qquad R_z(\phi) = \begin{bmatrix} \cos\phi & -\sin\phi & 0 \\ \sin\phi & \cos\phi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Combined, the upper 3x3 submatrix looks like this

$$\begin{aligned} R &= R_z(\phi)R_y(\theta)R_x(\psi) \\ &= \begin{bmatrix} \cos\theta\cos\phi & \sin\psi\sin\theta\cos\phi - \cos\psi\sin\phi & \cos\psi\sin\theta\cos\phi + \sin\psi\sin\phi \\ \cos\theta\sin\phi & \sin\psi\sin\theta\sin\phi + \cos\psi\cos\phi & \cos\psi\sin\theta\sin\phi - \sin\psi\cos\phi \\ -\sin\theta & \sin\psi\cos\theta & \cos\psi\cos\theta \end{bmatrix} \end{aligned}$$

- What if you're given a rotation matrix and want to know the angles?

⇨ There are *always at least two* sets of angles that produce the same rotation matrix [Slabaugh 2001]



Battle of the transformations: RT vs TR

Question: Matrix multiplication does not commute. What happens if transformations are applied in the wrong order?

Answer: The transformed object *can be* (and often is) at a different (incorrect) location.

Experiment: Transformation 1 vs. Transformation 2

Transformation 1

- Rotate 90 degrees about *y*
- Translate 1 unit along *z*

```
M.Translate(glm::vec3(0.0f, 0.0f, 1.0f));
M.Rotate(glm::vec3(0.0f, 1.0f, 0.0f), 90.0f);
RenderPerson();
```

Transformation 2

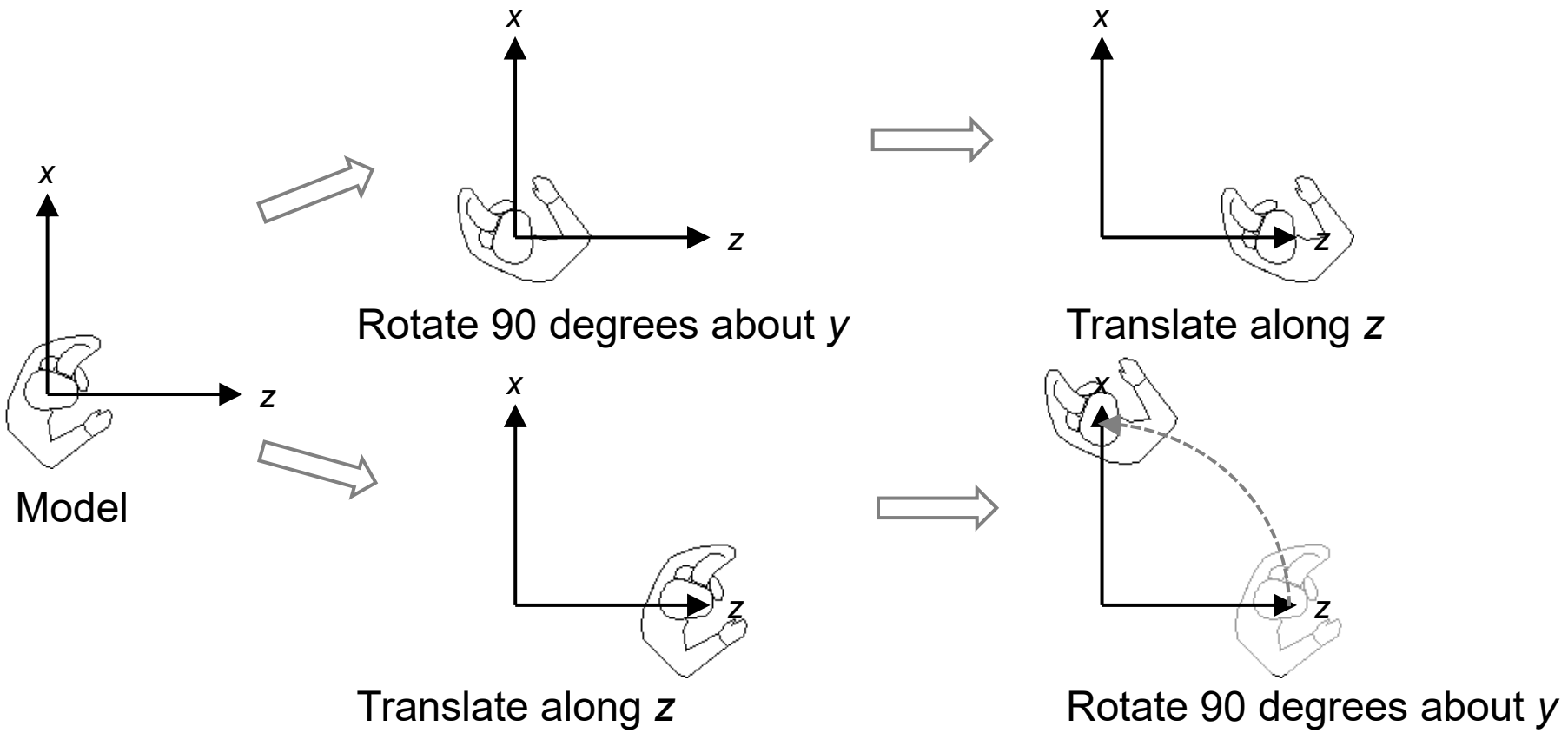
- Translate 1 unit along *z*
- Rotate 90 degrees about *y*

```
M.Rotate(glm::vec3(0.0f, 1.0f, 0.0f), 90.0f);
M.Translate(glm::vec3(0.0f, 0.0f, 1.0f));
RenderPerson();
```

⇨ One must think carefully about the sequence of transformations applied to an object

Battle of the transformations: RT vs TR

Experiment performed in class



Quaternions

- A *quaternion* simply is a vector of the form $[s, x, y, z]^T$, where s relates to an angle θ, and $[x, y, z]^T$ = **a** defines a unit axis to rotate about.

$$\mathbf{q} = \cos\frac{\theta}{2} + \mathbf{a}\sin\frac{\theta}{2}$$

- Quaternions provide an alternative way to model rotations.
- The mathematics of quaternions relies on *hypercomplex* numbers involving three imaginary numbers, which some find a bit exotic.
- GLM supports quaternions

```
#include "./include/glm/gtc/quaternion.hpp"

glm::quat q1 = glm::angleAxis(90.0f, glm::vec3(0.0f, 1.0f, 0.0f));
```

Converting from quaternions to a rotation matrix

- Often it is necessary to convert from a quaternion to a rotation matrix for use in an graphics API like OpenGL
- A unit quaternion **q** = $[s, x, y, z]^T$

has a corresponding 3x3 rotation matrix

$$\begin{bmatrix} 1-2y^2-2z^2 & 2xy-2sz & 2xz+2sy \\ 2xy+2sz & 1-2x^2-2z^2 & 2yz-2sx \\ 2xz-2sy & 2yz+2sx & 1-2x^2-2y^2 \end{bmatrix}$$

SLERP

- Often one uses quaternions in animation to interpolate between different orientations of an object
- One can linearly interpolate (LERP) between two quaternions $\mathbf{q}_1$ and $\mathbf{q}_2$

$$\mathbf{q} = (1-t)\mathbf{q}_1 + t\mathbf{q}_2$$

```
glm::mix(q1, q2, t)
normal behaviour for vectors
```

but the interpolated rotation does not necessarily maintain a constant rate, even when the interpolated quaternion is normalised.

- Alternatively, one can use spherical linear interpolation (SLERP)

$$\mathbf{q}(t) = \frac{\sin((1-t)\theta)}{\sin\theta}\mathbf{q}_1 + \frac{\sin(t\theta)}{\sin\theta}\mathbf{q}_2$$

```
glm::mix(q1, q2, t)
overloaded for quaternions
```

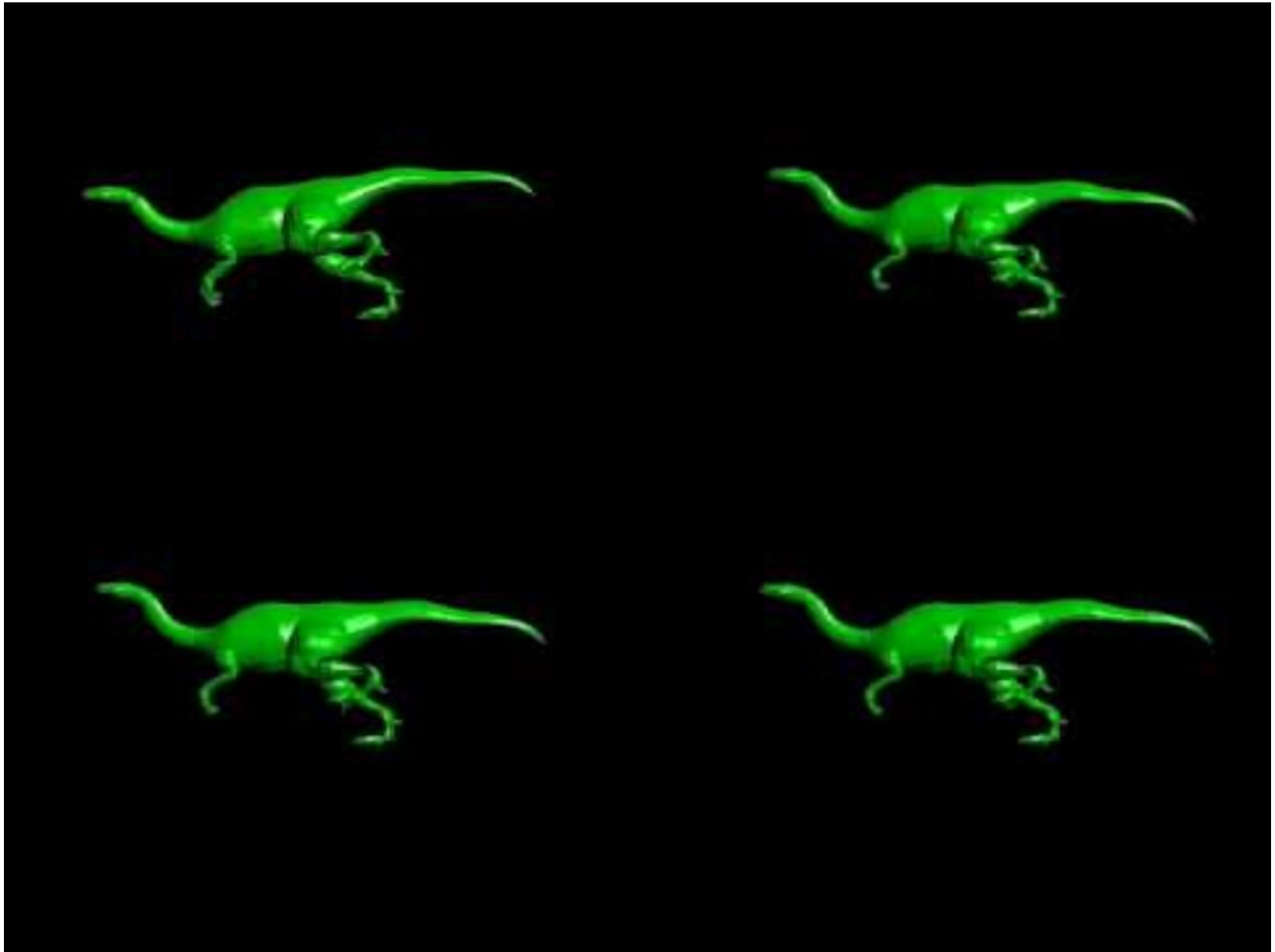
where θ is the angle between the two quaternions.

http://www.youtube.com/watch?v=94USA9yMzAw&feature=related

Rotation interpolation methods

Euler angle

Matrix



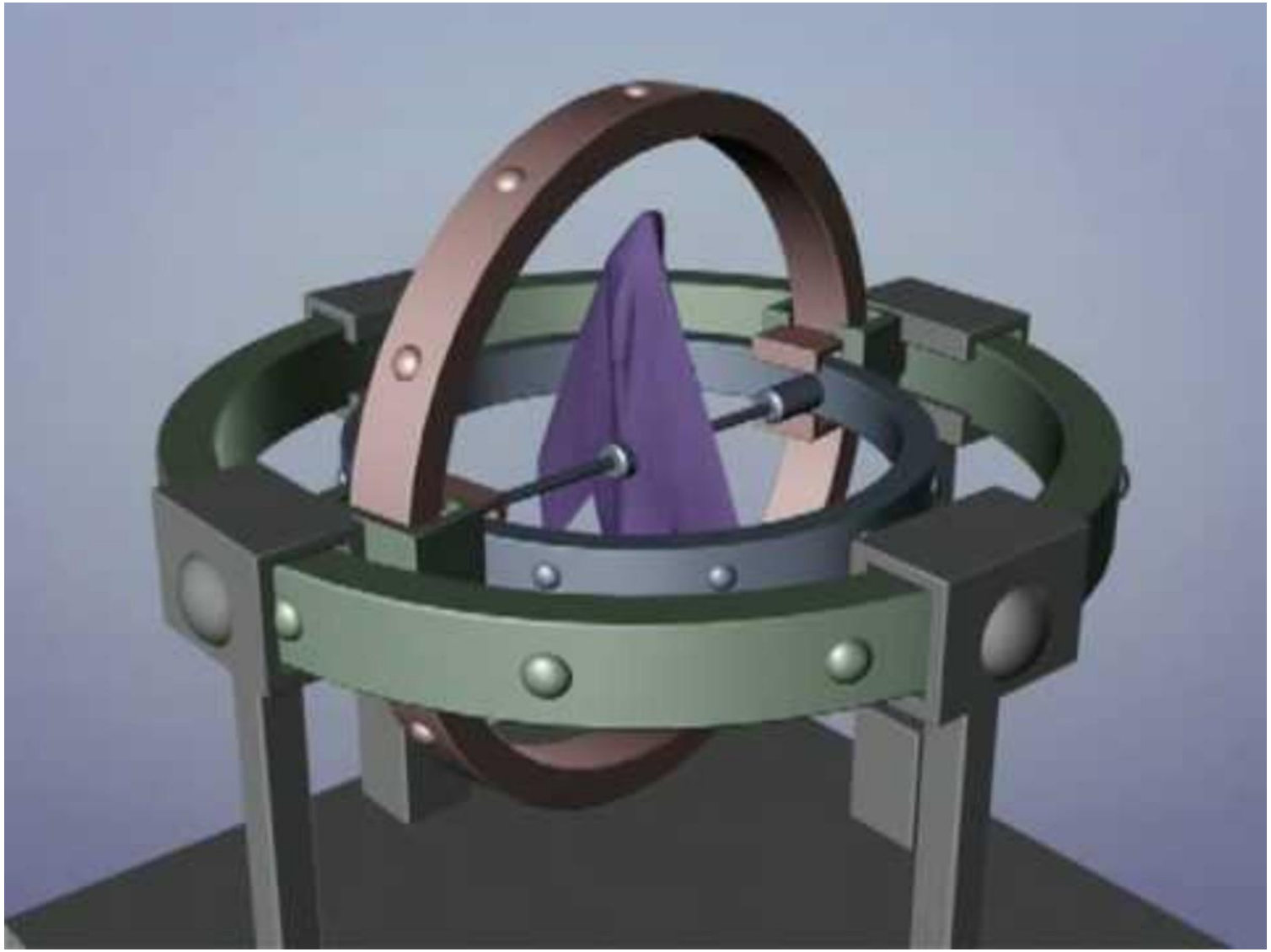
SLERP

Linear Quaternion interpolation

Example of SLERPING using GLM

```
#include "./include/glm/gtc/quaternion.hpp"


glm::vec3 p = glm::normalize(glm::vec3(1, 1, 1));
glm::quat q1 = glm::angleAxis(0.0f, glm::vec3(0.0f, 1.0f, 0.0f));
glm::quat q2 = glm::angleAxis(180.0f, p);

// Render an object using SLERP
modelViewMatrixStack.Push();
   modelViewMatrixStack.Translate(glm::vec3(0.0f, 2.0f, 150.0f));

   glm::quat q = glm::mix(q1, q2, t); // t between 0 and 1.  Mix overloaded to SLERP
   modelViewMatrixStack *= glm::mat4_cast(q); // Cast quaternion to mat4 to post-multiply

   modelViewMatrixStack.Scale(2.0f);
   pMainProgram->SetUniform("matrices.modelViewMatrix", modelViewMatrixStack.Top());
   pMainProgram->SetUniform("matrices.normalMatrix", m_pCamera-
         >ComputeNormalMatrix(modelViewMatrixStack.Top()));
   m_pSphere->Render();
modelViewMatrixStack.Pop();
```

Problems with Euler rotations – gimbal lock



https://www.youtube.com/watch?v=zc8b2Jo7mno&ab_channel=GuerrillaCG

Overview of today's lecture

Viewing
Vertex buffer objects
Trigonometry
Derivatives
Splines

Synthetic camera model

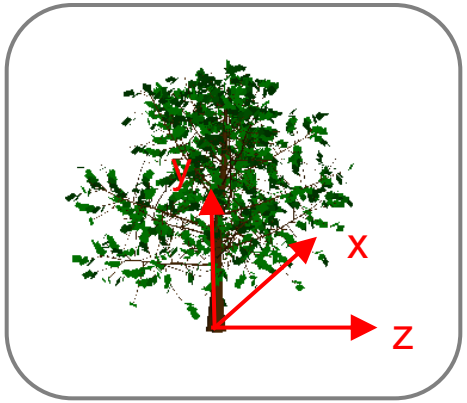
Imagine you are taking a picture with your camera

- Objects in the room may be best described in a local coordinate system unique to each object
- They also have position and orientation in the world
- Your camera also has a position and orientation, which determines where it is looking
- Your camera has settings, such as size, field of view, and resolution, which help define how the final picture is rendered

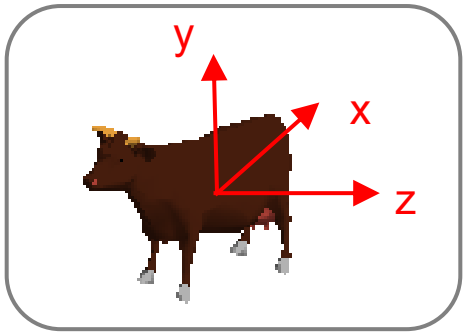
⇨ The synthetic camera model used in computer graphics mimics this process, using *transformations* between different *coordinate systems* to render an image. These transformations are implemented using matrices.

Object coordinates

Object coordinates describe a **local coordinate system** that is convenient for representing a 3D object. Each object (tree, cow) has its own object coordinate system



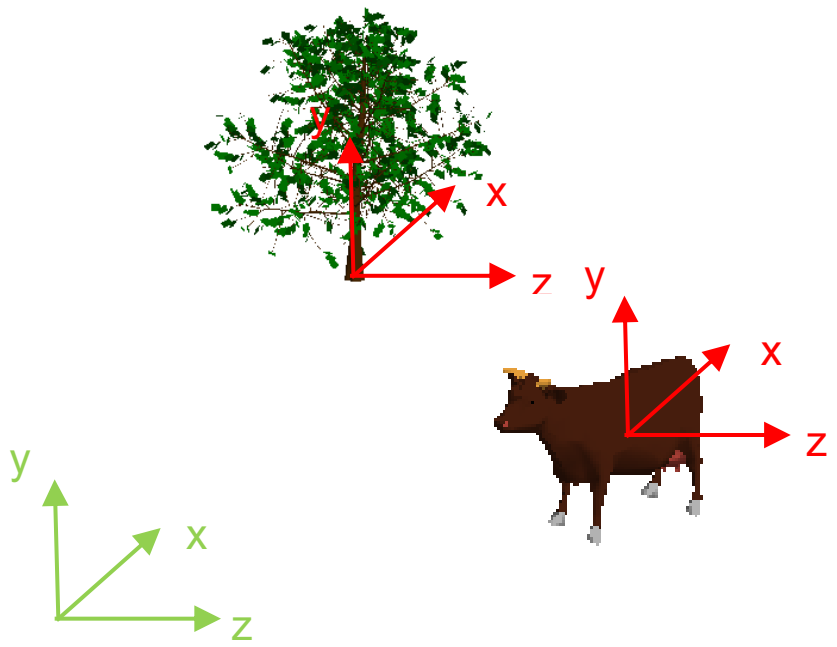
Object coordinate system 1



Object coordinate system 2

Modeling transformation

The modeling transformation places each object into the **world coordinate system**. Note there is only one world coordinate system.

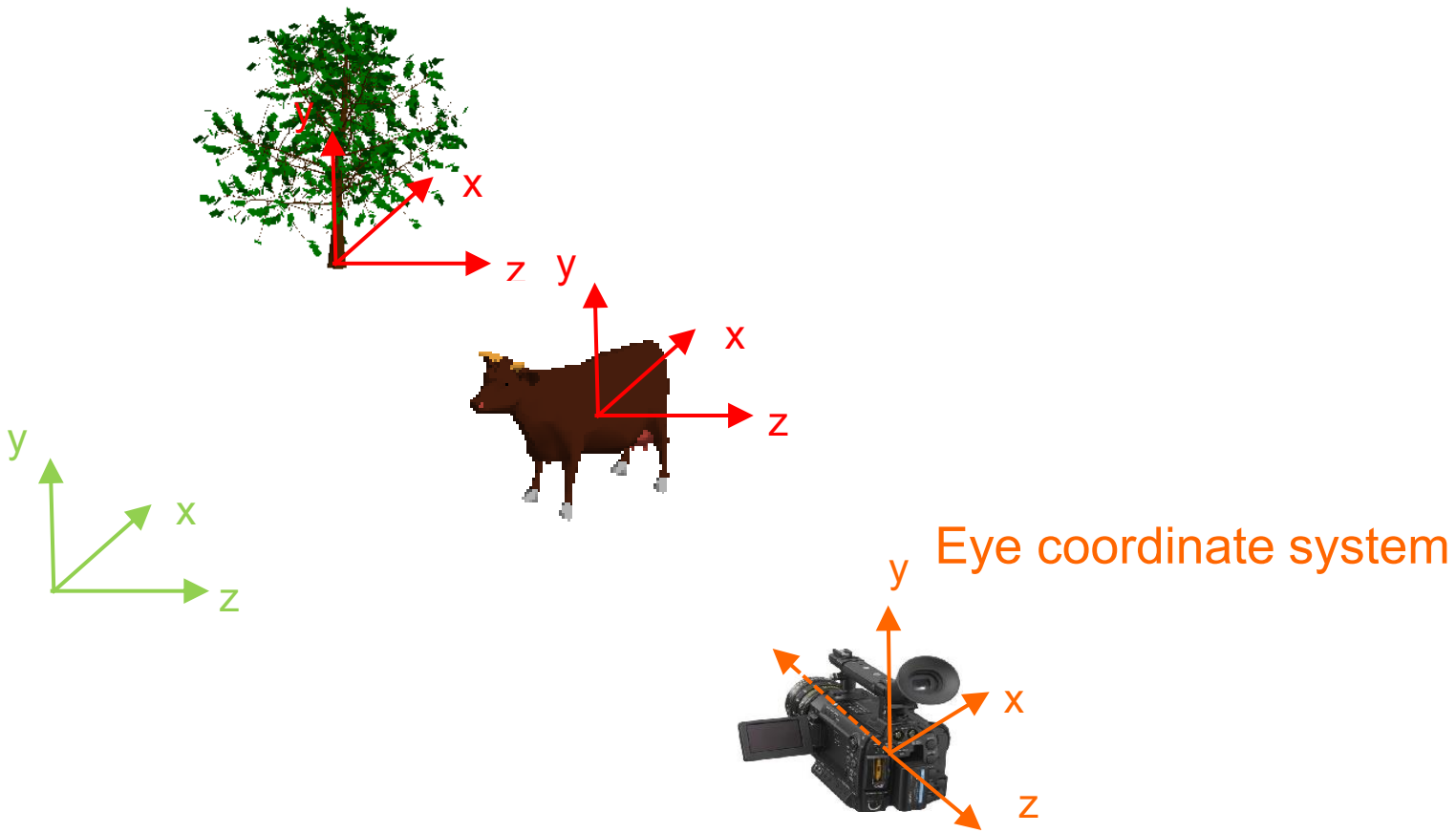


World coordinate system

In world coordinates, we can say the cow is so many units from the tree

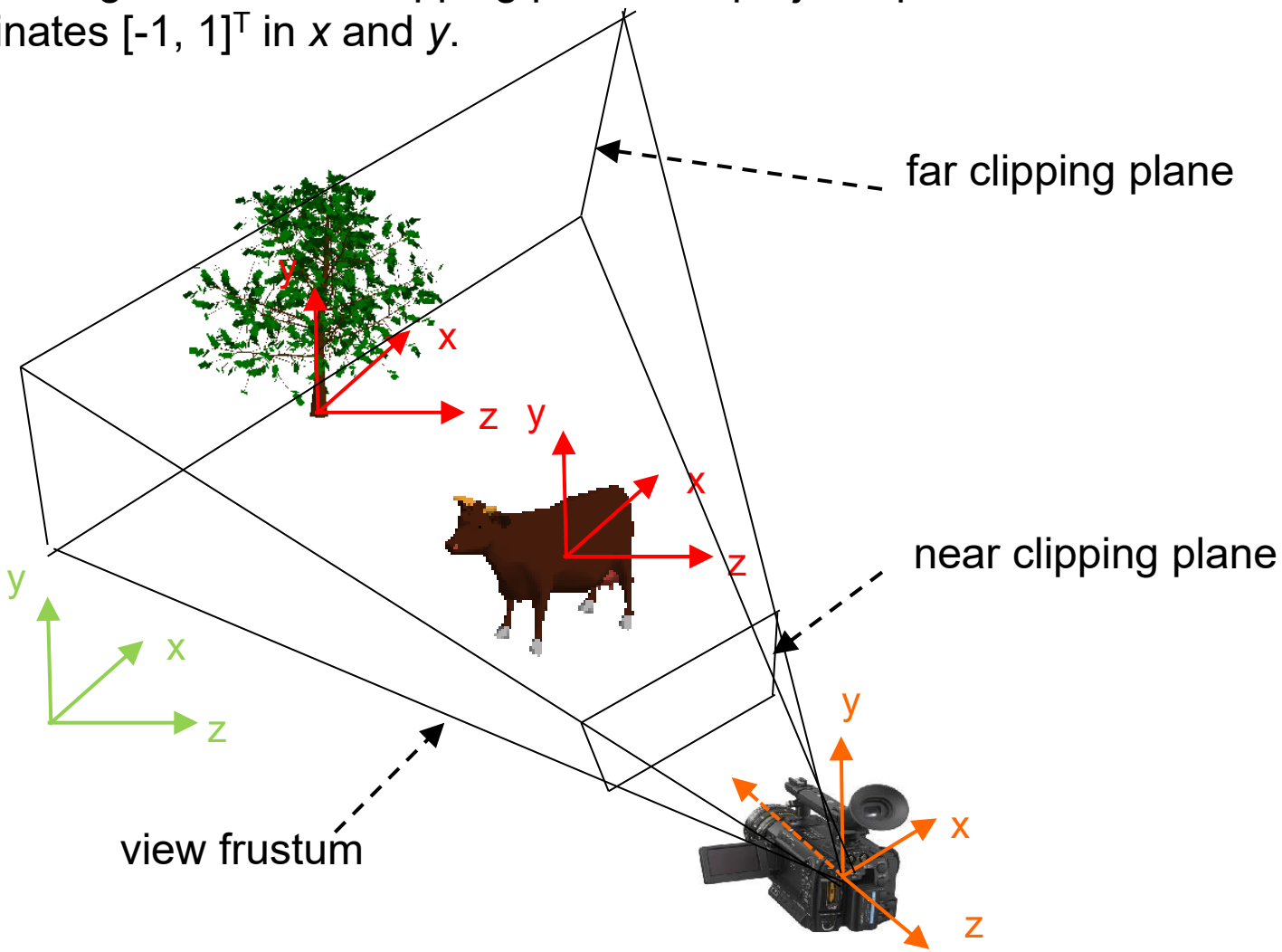
Viewing transformation

The viewing transformation specifies the location and orientation of the camera in world coordinates, and moves the scene into **eye coordinates** (also called camera coordinates)



Projection transformation

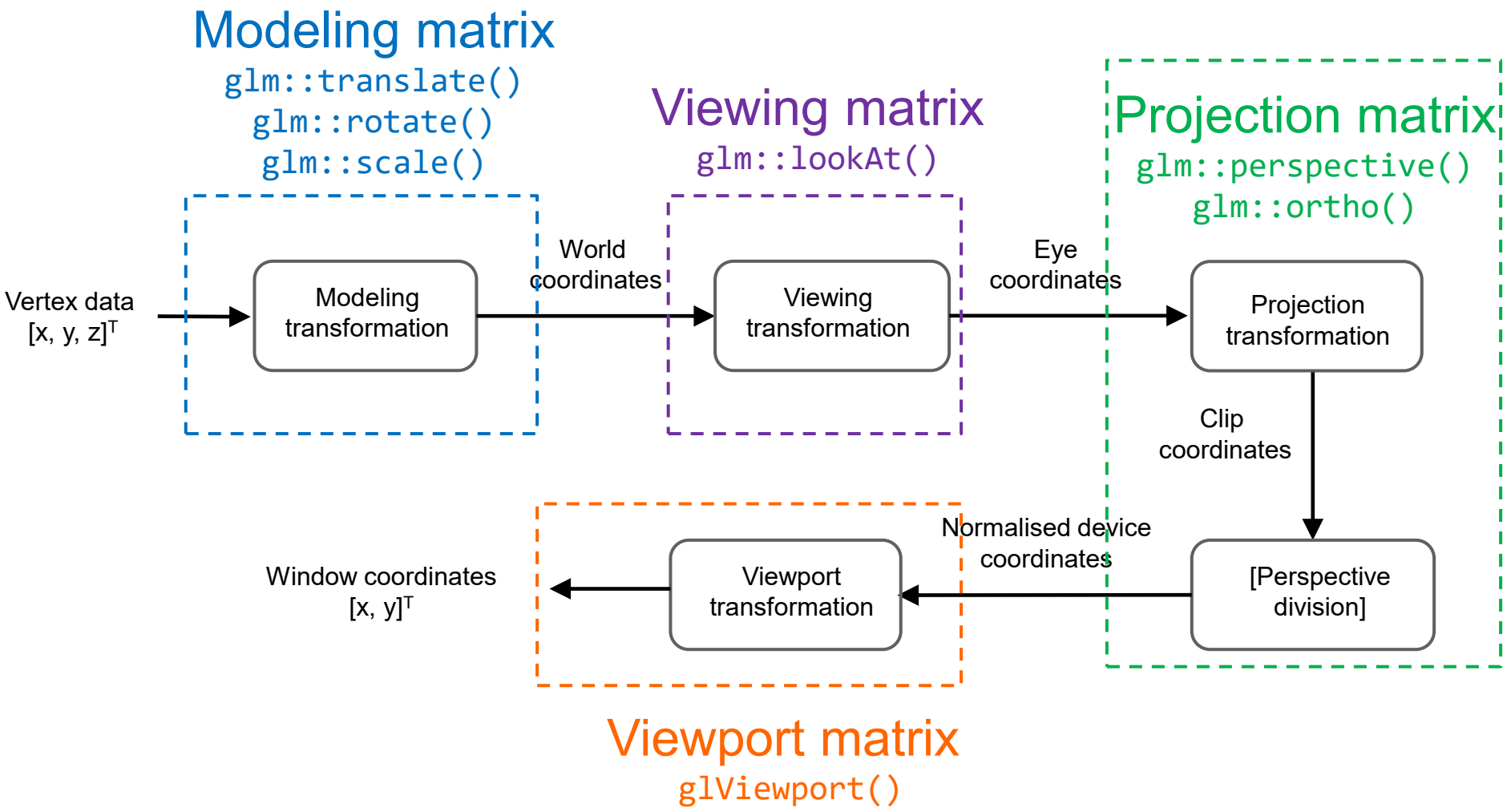
Defines the viewing volume and clipping planes. It projects points into normalised device coordinates $[-1, 1]^T$ in *x* and *y*.



Viewport transformation

Maps the projection of the scene into the two-dimensional window on the screen.

Block diagram

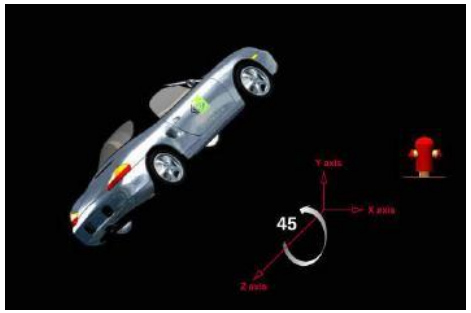


World coordinates

- Objects transformed to world coordinates using the modeling matrix

Translation

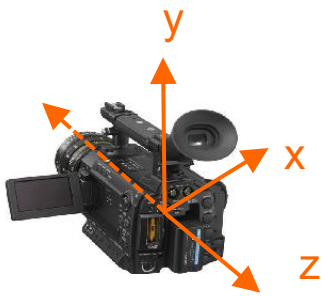
Rotation



Scaling

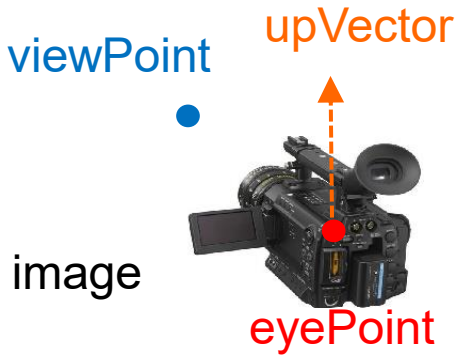
Eye coordinates

- Objects transformed to eye coordinates are positioned relative to the camera.
- The default camera placed at the origin and is oriented to look down the negative z-axis.
- The camera is implemented using a left handed coordinate system. However, all geometry is specified using a right handed coordinate system.



Positioning the camera

- There is a useful function, glm::`lookAt()`, that provides a simple interface for many viewing situations.
- It requires, in *world* coordinates,
  - A eye **point** representing in the camera position
  - A view **point** where the camera is looking
  - An up **vector** to consider "up" for the camera / rendered image
- It produces a 4x4 viewing matrix, which is used in a shader (typically vertex shader) to transform points.




```
glm::mat4 mViewingMatrix = glm::lookAt(vEye, vView, vUp);
```

⇨ To flythough a scene, change the eyepoint and redraw!

- Moving an object relative to the camera can have a similar effect as moving the camera relative to the object. Hence, modeling and viewing can combined in a single *modelview matrix*. This matrix is simply

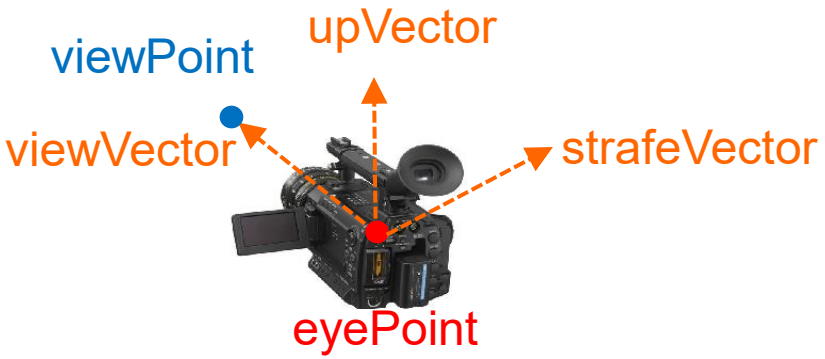
```
mModelviewMatrix = mViewingMatrix * mModelingMatrix
```

Camera vectors

- Given the data passed into glm::`lookAt()`, one can construct some useful vectors for viewing.

  - The viewVector = viewPoint – eyePoint
  - The strafeVector = viewVector x upVector
  - The orthogonal upVector = strafeVector x viewVector. This *may* be needed if the upVector is not orthogonal to the viewVector.

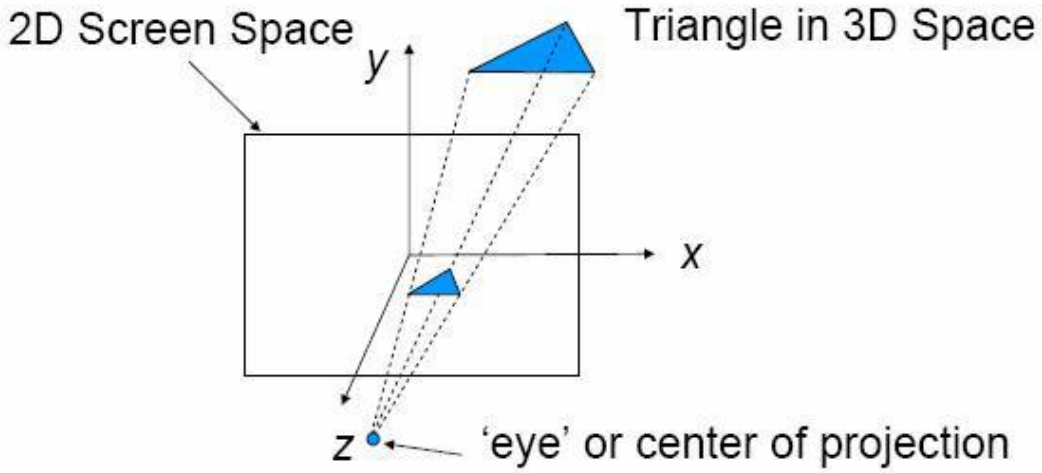
⇨ Typically, one normalises these vectors



- Moving the camera along the
  - viewVector makes the camera move forwards/backwards
  - strafeVector makes the camera move right/left
  - orthogonal upVector makes the camera move up/down

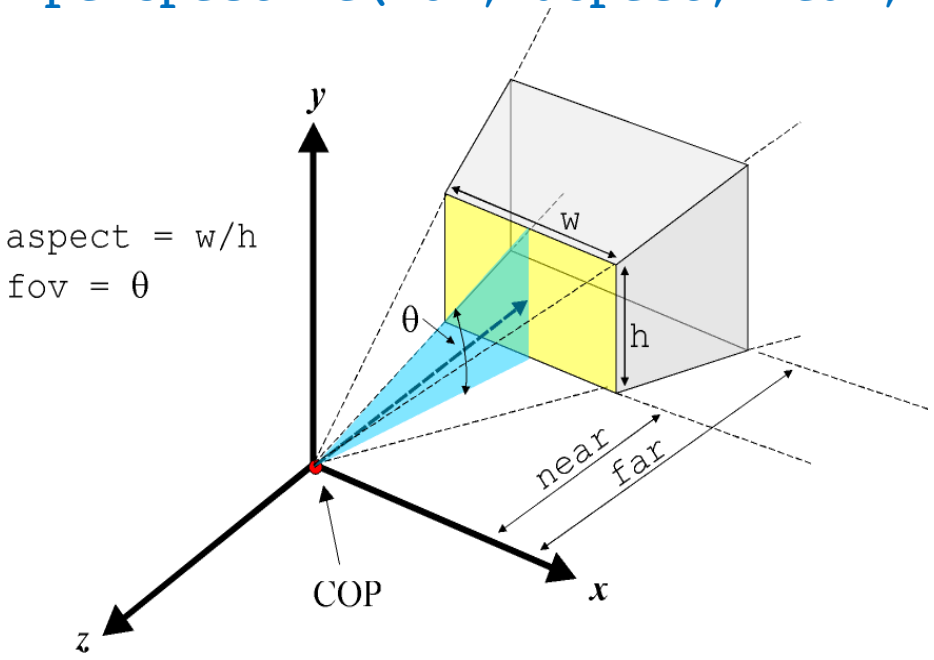
Perspective projection

- Similar to a real camera, objects farther away appear smaller than objects closer to the camera.
- Parallel lines converge at infinity.
- Typically used in 3D computer games.



Perspective projection

```
glm::perspective(fov, aspect, near, far);
```

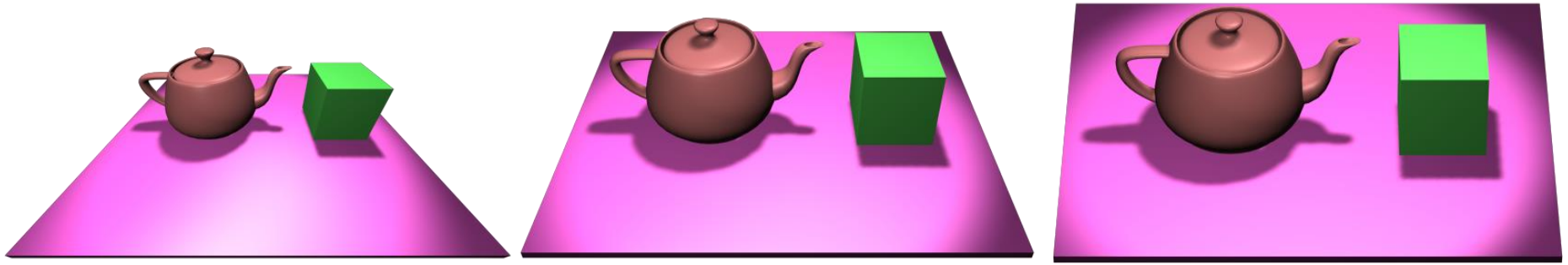


Example:

```
glm::mat4 mProjection = glm::perspective(45.0f, (GLfloat)width /
        (GLfloat)height, 1.0f, 150.0f);
```

This creates a viewing volume with a 45 degree field of view in the y direction (in eye coordinates) for the given window size (width, height). The near clipping plane is 1 unit away from the camera, and far clipping plane is 150 units away.

glm::perspective



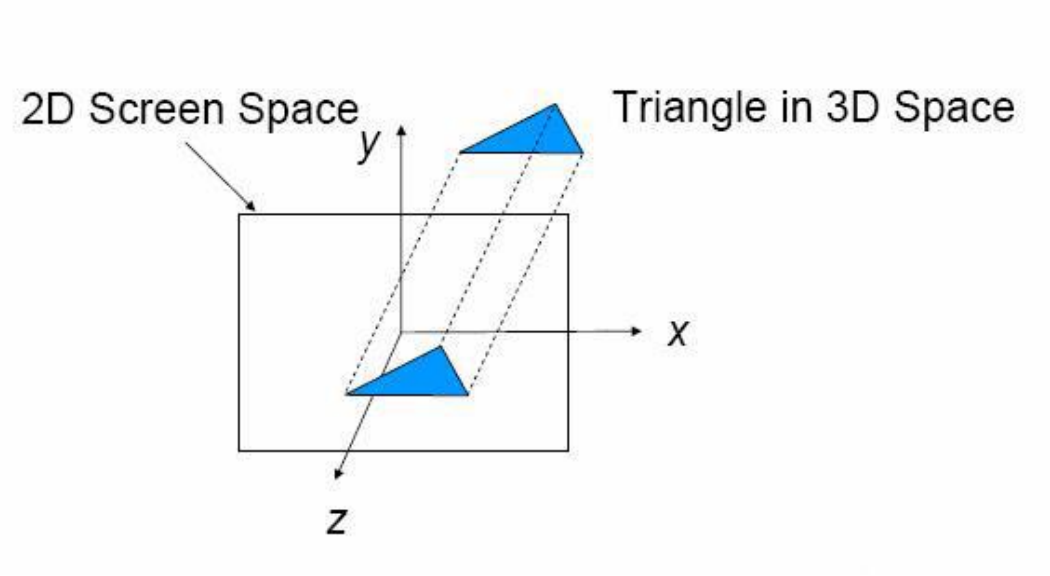
10mm Lens (fov = 122°) 35mm Lens (fov = 54°) 200mm Lens (fov = 10°)

- glm::perspective creates a 4x4 perspective projection matrix that is used in a shader (typically, vertex shader) to transform points.
- A few tips
  - Often, one sets up the projection transformation and does not change it during the course of a game, unless user changes the window
  - Think of `znear` and `zfar` as distance from the camera
  - A vertical field of view of 45 degrees is reasonable
  - The aspect ratio should match the window size
  - Don't use zero for `znear` (use a small number like 1.0 instead)
- Some OpenGL programmers combine the modelling, viewing, and projection matrices, and pass the combined matrix to the vertex shader.

```
mMVP = mProjectionMatrix * mViewingMatrix * mModelingMatrix
```

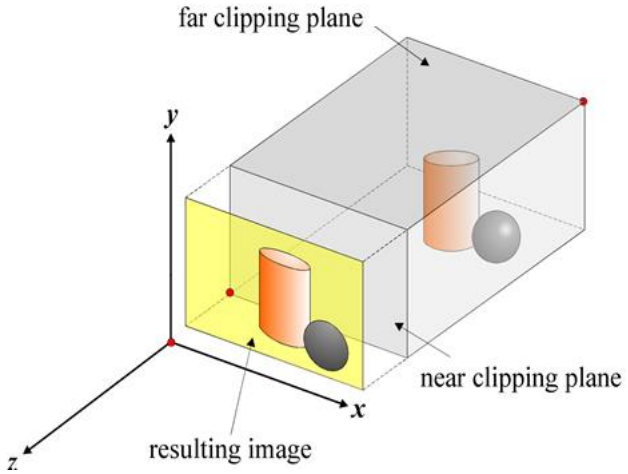
Orthographic projection

- Parallel lines remain parallel.
- Distance to camera does not affect rendering. Objects retain their size regardless of how far they are from the camera.
- Typically used in computer-aided design software and heads-up displays.



Orthographic projection in OpenGL

```
glm::ortho(xmin, xmax, ymin, ymax);
glm::ortho(xmin, xmax, ymin, ymax, zmin, zmax);
```



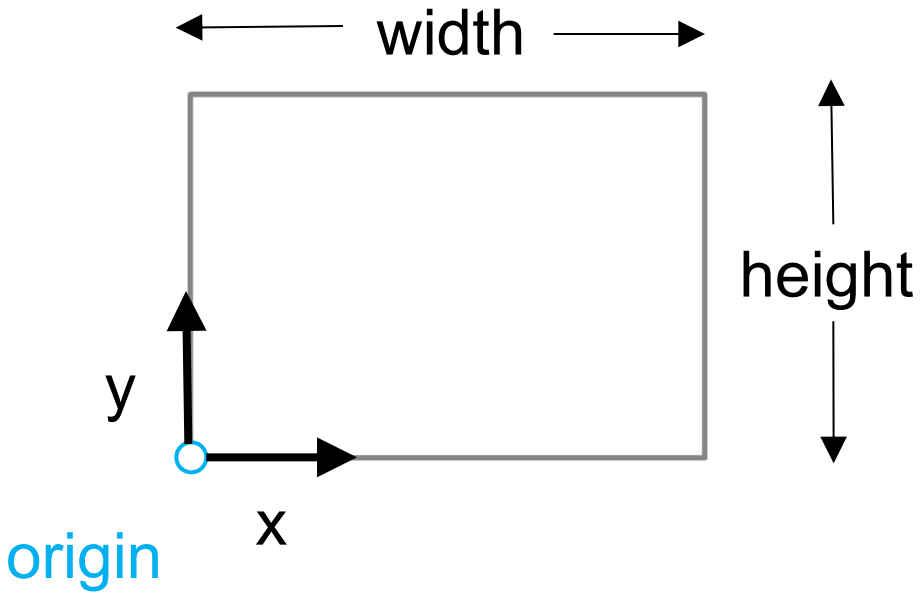

```
glm::mat4 mProjectionMatrix = glm::ortho(xmin, xmax, ymin, ymax)
```

- Specifies a logical 2D coordinate system which is to be mapped into the window positions indicated. Often one matches the coordinates to match the size of the window being rendered to.
- In the template `glm::ortho` is used for 2D rendering of text

- Perspective vs. orthographic: http://www.youtube.com/watch?v=s4IPY44eh2c

Viewport

- The viewport specifies the dimensions and of the 2D window where the rendering occurs
- `glViewport(x, y, width, height);`
- The arguments are in pixels
- The viewport is automatically set to the window size when the rendering context is created.
- Typically one sets the viewport whenever the window size changes.



Recap of viewing

The following transformations occur during viewing:

1. Object coordinates to world coordinates
   - Achieved using the modeling matrix
2. World coordinates to eye coordinates
   - Achieved with `glm::lookAt()` to set the viewing matrix based on camera location and orientation
3. Eye coordinates to normalised device coordinates
   - Achieved using a projection matrix defined by either
     - `glm::perspective`
     - `glm::ortho`
4. Normalised device coordinates to display coordinates
   - Achieved using `glViewport` to define a mapping from normalised device coordinates to display coordinates

Putting it all together

- Initialisation
  - Create window that has size width, height
  - Use WGL to initialise OpenGL
    - `SetPixelFormat(...)`
    - `wglCreateContextAttribsARB(...)` Also when the window is resized
    - `wglMakeCurrent(...)`
  - Set up viewport
    - `glViewport(0, 0, width, height);`
  - Set up the projection matrix
    - `glm::perspective(45.0f, (GLfloat)width/(GLfloat)height, 1, 150.0f);`
  - Load shaders and other resources, compile, pass to GPU
  - Pass geometry to GPU

⇨ Typically the modeling, viewing, and projection matrices are used in the **vertex shader** to transform points.

Putting it all together

- Rendering a frame
  - Clear the buffers
    - `glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);`
  - Set the view (or modelview) matrix
    - `glm::lookAt();`
  - Set the model (or modelview) matrix
    - `glm::translate(), glm::rotate(), glm::scale()`
  - Pass matrices to shader
  - Render object
    - `glDrawArrays()`
  - Swap buffers to display the rendered frame

⇨ Just with this you could make a game!

Matrix stack

- If you're using a matrix stack object, equivalent methods exist:

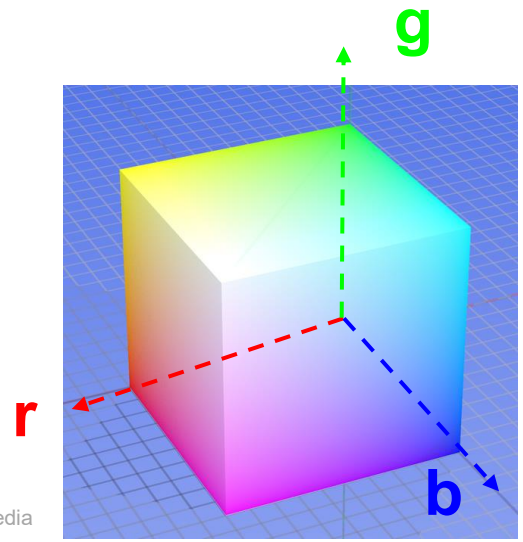
glm::translate()	Translate()
glm::rotate()	Rotate()
glm::scale()	Scale()
glm::lookAt()	LookAt()
glm::ortho	Orthographic()
glm::perspective	Perspective()
(more)	(more)

```
// Render the horse
modelViewMatrixStack.Push();
   modelViewMatrixStack.Translate(glm::vec3(0.0f, 0.0f, 0.0f));
   modelViewMatrixStack.Rotate(glm::vec3(0.0f, 1.0f, 0.0f), 180.0f);
   modelViewMatrixStack.Scale(2.5f);
   ...
modelViewMatrixStack.Pop();
```

RGB colour

- Colour in OpenGL is typically represented as RGB
- RGB is a colour space
  - Red axis, green axis, blue axis
  - Values typically in the range from 0.0 (none) to 1.0 (full colour) along each axis
  - Point in the colour space can be expressed as a vector

$$\mathbf{C} = \begin{bmatrix} r \\ g \\ b \end{bmatrix}$$

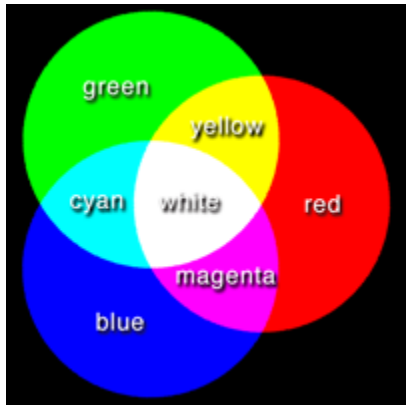


Source: wikipedia

RGB colour

RGB uses additive colour mixing, because it describes what kind of *light* needs to be *emitted* to produce a given colour. Light is added together to create form from out of the darkness.

Colour Component R	G	B	Colour Common Name
0	0	0	Black
0	0	1	Blue
0	1	0	Green
0	1	1	Cyan
1	0	0	Red
1	0	1	Magenta
1	1	0	Yellow
1	1	1	White



Vertex attributes

- A vertex *may* have several attributes:
  - Position
  - Colour
  - Texture coordinate (to be discussed… stay tuned)
  - Normal (to be discussed… stay tuned)

- Two vertices are equivalent if and only if all their attributes are equal.
- Consider the following two vertices, **p** and **q**:

p.x = 0	q.x = 0
p.y = 1	q.y = 1
p.z = 0	q.z = 0
p.r = 1	q.r = 0
p.g = 0	q.g = 1
p.b = 0	q.b = 0

- Are these vertices equal?
- No. **p** ≠ **q**. While they have the same position, their colours are different.

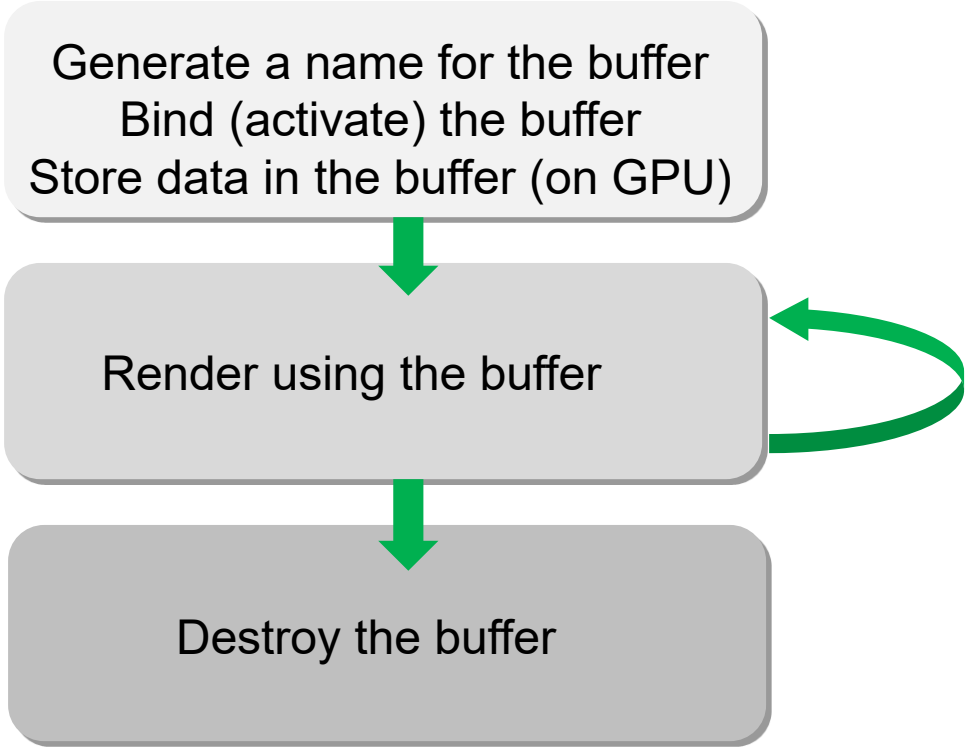
Immediate mode vs retained mode

- When the data to be rendered is stored in RAM on the client side, the rendering is called *immediate mode*, because the client is asking the server to render the data immediately after it is sent to the server. Former examples of immediate mode rendering include
  - `glBegin... glEnd`: pass vertices to GPU forming one primitive at a time
  - Vertex arrays: pass vertices to GPU in a single call (for each frame)
- Rendering with `glBegin…glEnd` is particularly slow **because the graphics hardware must wait** for each primitive to be sent before it can be rendered!
- In contrast, *retained mode* rendering passes geometry stores the vertices in VRAM on the GPU. Rendering is often much more efficient.

⇨ Immediate mode rendering was deprecrated in OpenGL 3.0 and dropped from the core in OpenGL 3.1.

Vertex buffer objects (VBOs)

- In modern OpenGL, vertex attribute data is stored in a buffer on the GPU.
- A *buffer object* is an OpenGL container used to store generic data, and a *vertex buffer object (VBO)* is one that stores vertex attributes.
- VBOs store per-vertex data that is used by the vertex shader during rendering.
- To use a VBO, one typically applies the following steps:

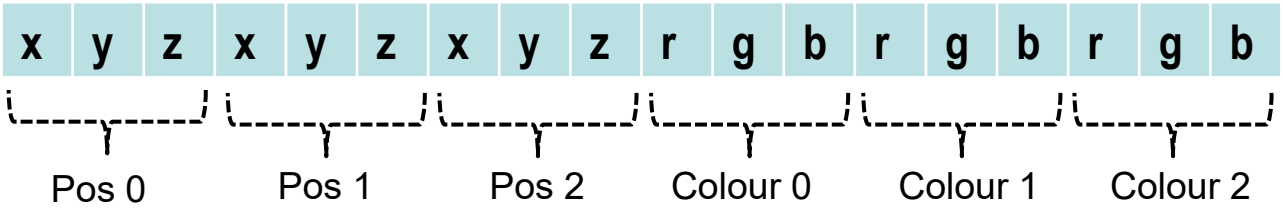


Using a VBO with tightly packed attributes

- Let's revisit our multi-coloured triangle.

```
// Setup triangle vertex positions
const GLfloat fTrianglePositions[] = {-1.0f, 0.0f, 0.0f, // Left
                                       1.0f, 0.0f, 0.0f, // Right
                                       0.0f, 1.0f, 0.0f}; // Up
const GLfloat fTriangleColours[] = {0.0f, 1.0f, 0.0f, // Green
                                       1.0f, 0.0f, 0.0f, // Red
                                       0.0f, 0.0f, 1.0f}; // Blue
```

- We're going to put all this data into a single VBO, that will look like this:



Initialisation

```
// Generate a VAO and bind it
glGenVertexArrays(1, &m_uiVAO);
glBindVertexArray(m_uiVAO);

// Create a VBO and bind it
glGenBuffers(1, &uiVBO);
glBindBuffer(GL_ARRAY_BUFFER, uiVBO);

// Allocate space for the VBO
glBufferData(GL_ARRAY_BUFFER, sizeof(fTrianglePositions) + sizeof(fTriangleColours), NULL, GL_STATIC_DRAW);

// Pass individual arrays into the VBO.  First positions, then colours
glBufferSubData(GL_ARRAY_BUFFER, 0, sizeof(fTrianglePositions), fTrianglePositions);
glBufferSubData(GL_ARRAY_BUFFER, sizeof(fTrianglePositions), sizeof(fTriangleColours), fTriangleColours);

// Enable position in the 0th vertex attribute
glEnableVertexAttribArray(0);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, 0);

// Enable colour in the 1st vertex attribute
glEnableVertexAttribArray(1);
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 0, (const GLvoid *) sizeof(fTrianglePositions));
```

Initialisation: OpenGL functions

- `void glGenBuffers(GLsizei n, GLuint *buffers)` generates n buffers stored in VRAM on the GPU.
- `void glBindBuffer(GLenum target, GLuint buffer)` binds the buffer, making it active. The target is `GL_ARRAY_BUFFER` for a VBO.
- `void glBufferData(GLenum target, GLsizeiptr size, const GLvoid *data, GLenum usage)` can be used to fill the currently bound buffer. However, if data is NULL, no data is copied – instead, the buffer is allocated.
- `void glBufferSubData(GLenum target, GLintptr offset, GLsizeiptr size, const GLvoid *data)` is used to replace part of the previously allocated buffer. The data will be copied at offset bytes in the array.
- `void glEnableVertexAttribArray(GLuint index)` enables a generic vertex attribute arrays. We will use one for position and another for colour. This is so the vertex shader knows where to find the position and colours in the array.
- `void glVertexAttribPointer(GLuint index, GLint size, GLenum type, GLboolean normalised, GLsizei stride, const GLvoid* pointer)` indicates where the vertex attributes (positions, colours) are located in the array. Size is the number of attributes per vertex. When stride is 0 since the attributes are *tightly packed*.

Rendering

- Client program:

```
// Bind the VAO and call glDrawArrays
glBindVertexArray(m_uiVAO);
glDrawArrays(GL_TRIANGLES, 0, 3);
```

  - `void glDrawArrays(GLenum mode, GLint first, GLsizei count)` renders primitives, using *count* vertices starting at *first*.

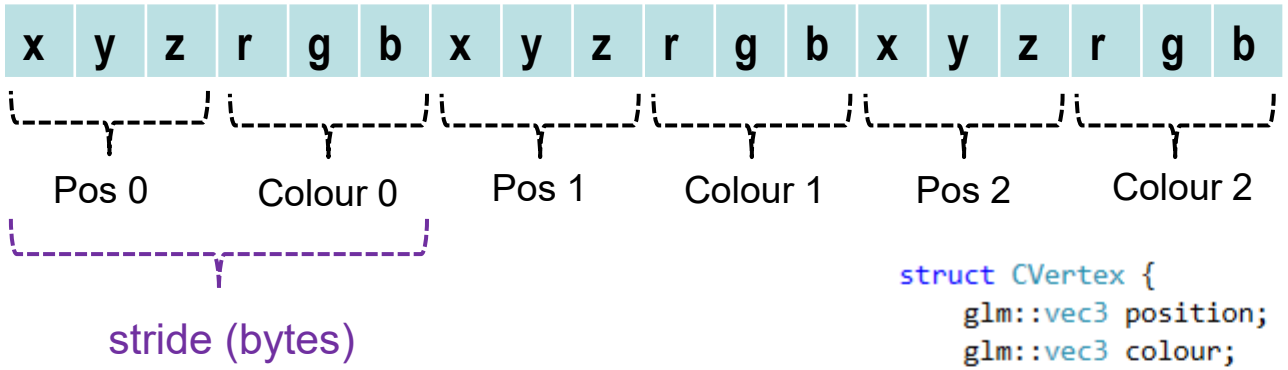
- Vertex shader:

```
#version 400

uniform mat4 modelMatrix;
uniform mat4 viewMatrix;
uniform mat4 projectionMatrix;

uniform float t;

layout (location = 0) in vec3 inPosition;
layout (location = 1) in vec3 inColour;

out vec3 vColour;

void main()
{
    gl_Position = projectionMatrix * viewMatrix * modelMatrix * vec4(inPosition, 1.0);
    vColour = inColour;
}
```

coming from VBO

Using a VBO with interleaved attributes

- Let's render the same triangle, but this time use *interleaved attributes*, like this:




```
struct CVertex {
    glm::vec3 position;
    glm::vec3 colour;
};

std::vector<CVertex> vertices;
CVertex v0, v1, v2;
v0.position = glm::vec3(-1, 0, 0);
v0.colour = glm::vec3(0, 1, 0);
v1.position = glm::vec3(1, 0, 0);
v1.colour = glm::vec3(0, 0, 1);
v2.position = glm::vec3(0, 1, 0);
v2.colour = glm::vec3(1, 0, 0);
vertices.push_back(v0);
vertices.push_back(v1);
vertices.push_back(v2);
```

- Interleaved attributes are often more convenient

Passing the data to the GPU

```
// Generate a VAO and bind it
glGenVertexArrays(1, &m_uiVAO);
glBindVertexArray(m_uiVAO);

// Create a VBO and bind it
glGenBuffers(1, &uiVBO);
glBindBuffer(GL_ARRAY_BUFFER, uiVBO);

// Pass data to GPU
glBufferData(GL_ARRAY_BUFFER, vertices.size() * sizeof(CVertex), &vertices[0], GL_STATIC_DRAW);

// Enable position in the 0th vertex attribute
glEnableVertexAttribArray(0);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, sizeof(CVertex), 0);

// Enable colour in the 1st vertex attribute
glEnableVertexAttribArray(1);
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, sizeof(CVertex), (const GLvoid *) sizeof(glm::vec3));
```

stride

- `void glVertexAttribPointer(GLuint index, GLint size, GLenum type, GLboolean normalised, GLsizei stride, const GLvoid* pointer)` indicates where the vertex attributes (positions, colours) are located in the array.
  - The first vertex position is at 0 bytes in the array. After this position, we must skip *stride* bytes to get to the next position.
  - The first colour position is at `sizeof(glm::vec3)` bytes in the array. After this colour, we **must skip stride bytes** to get to the next colour.

Vertex array objects (VAOs)

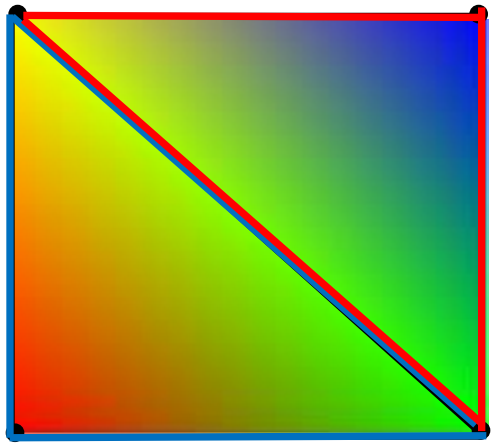
- VAOs are useful, particularly for non-trivial applications that use multiple VBOs. VAOs are a “wrapper” around a VBO, managing state information. This makes VBOs easier and more efficient to use.
- The following functions are used with VAOs:
  - `void glGenVertexArrays(GLsize n, GLuint *arrays)`: creates *n* VAOs.
  - `void glBindVertexArray(GLuint array)`: binds a VAO.
  - `void glDeleteVertexArray(GLuint array)`: deletes a VAO.

⇨ Tip: *Always* use a VAO in conjunction with your VBO

Indexed rendering

- So far we have discussed the GL_ARRAY_BUFFER binding. Another related buffer binding is the GL_ELEMENT_ARRAY_BUFFER.
- An element array buffer stores indices of vertices, and is typically rendered using glDrawElements. Consider rendering the following quad:

V[3]=[-1,1,0]^T V[2]=[1,1,0]^T



V[0]=[-1,-1,0]^T V[1]=[1,-1,0]^T

Triangle 1: vertex indices 0, 1, 3
Triangle 2: vertex indices 1, 2, 3

```
std::vector<CVertex> vertices;
CVertex v0, v1, v2, v3;
v0.position = glm::vec3(-1, -1, 0);
v0.colour = glm::vec3(1, 0, 0);
v1.position = glm::vec3(1, -1, 0);
v1.colour = glm::vec3(0, 1, 0);
v2.position = glm::vec3(1, 1, 0);
v2.colour = glm::vec3(0, 0, 1);
v3.position = glm::vec3(-1, 1, 0);
v3.colour = glm::vec3(1, 1, 0);
vertices.push_back(v0);
vertices.push_back(v1);
vertices.push_back(v2);
vertices.push_back(v3);

std::vector<GLuint> indices;
indices.push_back(0);
indices.push_back(1);
indices.push_back(3);
indices.push_back(1);
indices.push_back(2);
indices.push_back(3);
```

Two triangles with glDrawElements

setup

```
// Generate a VAO and bind it
glGenVertexArrays(1, &m_uiVAO);
glBindVertexArray(m_uiVAO);

// Create a VBO and bind it
glGenBuffers(1, &uiVBOVertices);
glBindBuffer(GL_ARRAY_BUFFER, uiVBOVertices);

// Pass data to GPU
glBufferData(GL_ARRAY_BUFFER, vertices.size() * sizeof(CVertex), &vertices[0], GL_STATIC_DRAW);

// Create buffer for indices and bind it
glGenBuffers(1, &uiVBOIndices);
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, uiVBOIndices);

// Pass indices to GPU
glBufferData(GL_ELEMENT_ARRAY_BUFFER, indices.size() * sizeof(GLuint), &indices[0], GL_STATIC_DRAW);

// Enable position in the 0th vertex attribute
glEnableVertexAttribArray(0);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, sizeof(CVertex), 0);

// Enable colour in the 1st vertex attribute
glEnableVertexAttribArray(1);
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE,  sizeof(CVertex), (const GLvoid *) sizeof(glm::vec3));
```

rendering

```
// Bind the VAO and call glDrawElements
glBindVertexArray(m_uiVAO);
glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_INT, 0);
```

Buffer object usage models & unbinding

- The usage parameter in `glBufferData` is a performance hint to help the OpenGL driver allocate memory in the correct location in VRAM.
  - GL_STATIC_DRAW: Buffer contents static but used frequently
  - GL_DYNAMIC_DRAW: Buffer contents changing and used frequently

- To unbind a VBO, bind to 0 (null)
  - `glBindBuffer(GL_ARRAY_BUFFER, 0);`
  - `glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, 0);`

- To unbind a VAO, bind to 0 (null)
  - `glBindVertexArray(0);`

Helper classes in the OpenGL template

- In the OpenGL template, there are **two** helper classes called `CVertexBufferObject` and `CVertexBufferObjectIndexed` that simplify usage of VBOs. They both provide generic containers to store data and upload to the GPU.
  - `CVertexBufferObject` is for non-indexed drawing (`glDrawArrays`)
  - `CVertexBufferObjectIndexed` is for indexed drawing (`glDrawElements`)

```
CVertexBufferObject vbo; // Instantiates object
vbo.Create();  // Creates a VBO
vbo.Bind();  // Binds the VBO
vbo.AddData(…); // Adds data to the VBO (position, colour, etc.)
vbo.UploadDataToGPU(hint); // Sends data to the GPU using hint
```

⇨ Examples of usage in CPlane and CSphere classes

Example in OpenGL template (CPlane)

```
glGenVertexArrays(1, &m_vao);
glBindVertexArray(m_vao);
m_vbo.Create();
m_vbo.Bind();
// Set data (not shown)
for (int i = 0; i < 4; i++) {
    m_vbo.AddData(&vPlaneVertices[i], sizeof(glm::vec3));
    m_vbo.AddData(&vPlaneTexCoords[i], sizeof(glm::vec2));
    m_vbo.AddData(&vPlaneNormal, sizeof(glm::vec3));
}
m_vbo.UploadDataToGPU(GL_STATIC_DRAW);

GLsizei istride = 2*sizeof(glm::vec3)+sizeof(glm::vec2);
glEnableVertexAttribArray(0);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, istride, 0);
glEnableVertexAttribArray(1);
glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, istride, (GLvoid*)sizeof(glm::vec3));
glEnableVertexAttribArray(2);
glVertexAttribPointer(2, 3, GL_FLOAT, GL_FALSE, istride,
                            (GLvoid*)(sizeof(glm::vec3)+sizeof(glm::vec2)));
```

Rendering primitives

Method	OpenGL version it became core	Deprecated in OpenGL >=3.0?	Available in OpenGL ES 3.0?	Available in WebGL 1.0?
`glBegin()... glEnd()`	1.0	**Deprecated**	**No**	**No**
Display lists	1.0	**Deprecated**	**No**	**No**
Vertex arrays	1.1	**Deprecated**	Yes	Yes
VBO	1.5	**Not deprecated**	Yes	Yes
VAO	3.0	**Not deprecated**	Yes	Yes (extension)

⇨ Current OpenGL guidelines recommend rendering all primitives using VBOs / VAOs.

Demo

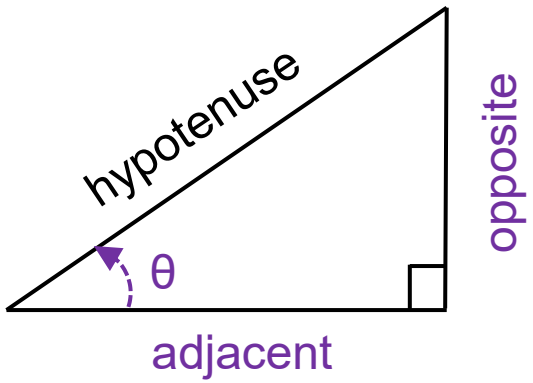
Dragon dataset: 871k triangles
Generated by Stanford U. using laser range scanning

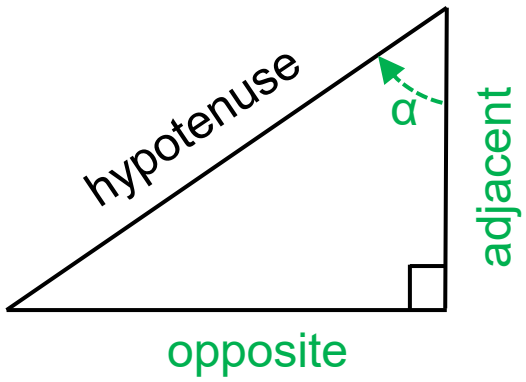
Rendering method	Frame rate (FPS)*
`glBegin()...glEnd()`	10-30
Vertex arrays	230+
Vertex buffer objects	230+

*on my laptop: Nvidia Geforce GTX 3080

Trigonometry

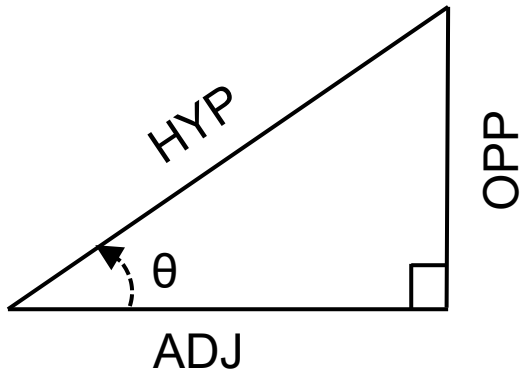
- A branch of mathematics that studies triangles, the relationships between their sides, and the angles between these sides.
- Trigonometric functions are often used to describe these relationships.
- Trigonometry has many fundamental applications, including angles, rotations, and more (e.g., Fourier techniques).
- Consider a right triangle as below:
  - The longest side is called the *hypotenuse*, and is opposite the right angle.
  - For an angle other than the right angle, we can define *adjacent* and *opposite* sides.





Sine, cosine, tangent

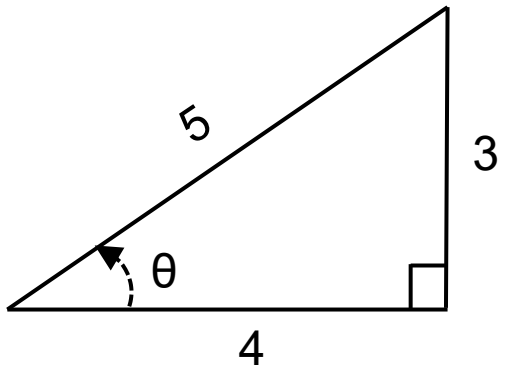
- Sine, cosine, and tangent are trigonometric functions that relate to the edge lengths of the triangle. Let the length of the hypotenuse, adjacent, and opposite sides be HYP, ADJ, and OPP respectively.
- Then,
  - $\sin(\theta) = \dfrac{\text{OPP}}{\text{HYP}}$
  - $\cos(\theta) = \dfrac{\text{ADJ}}{\text{HYP}}$
  - $\tan(\theta) = \dfrac{\text{OPP}}{\text{ADJ}}$



- Some people use the mnemonic SOH CAH TOA – meaning **s**ine is **o**pposite over **h**ypotenuse, **c**osine is **a**djacent over **h**ypotenuse, **t**angent is **o**pposite over **a**djacent.

Example

- Consider the following triangle:



  - $\sin(\theta) = \frac{3}{5} = 0.6$
  - $\cos(\theta) = \frac{4}{5} = 0.8$
  - $\tan(\theta) = \frac{3}{4} = 0.75$

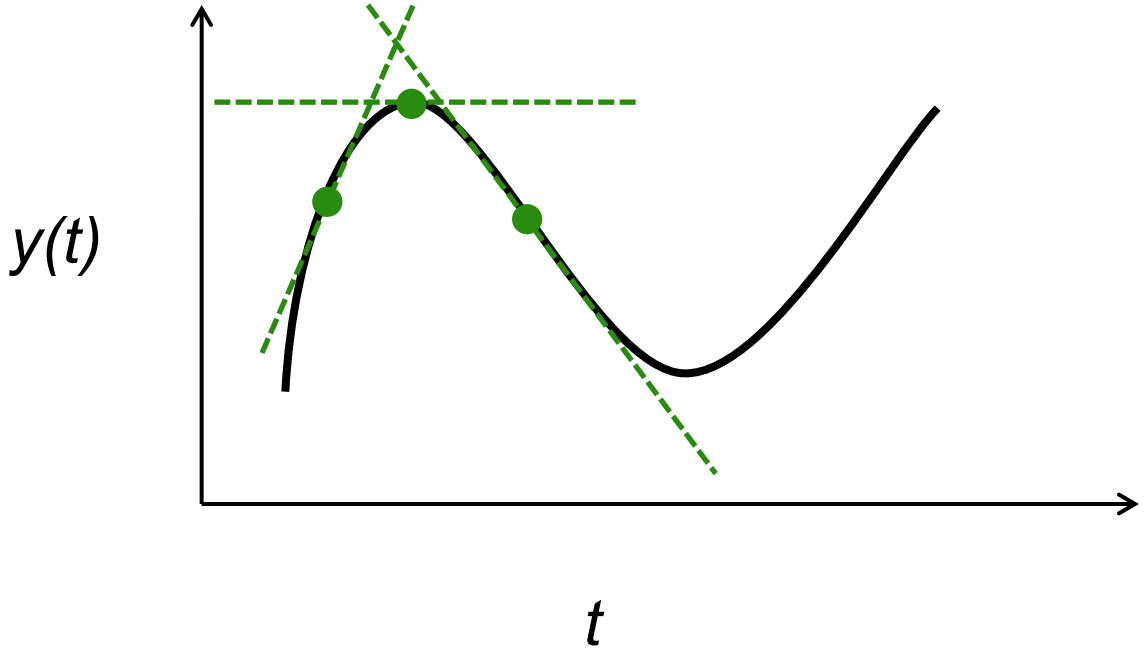
- To determine θ, simply apply the inverse trigonometric function:

$$\theta = \tan^{-1}\left(\frac{3}{4}\right) = 36.8^\circ$$

- In code, you might write
  - `theta = atan(opp / adj);` or **`theta = atan2(opp, adj);`**
  - Note: these functions return the angle in *radians*. To convert to degrees, simply multiply by (`180.0f / M_PI`). Atan returns an angle defined on a 180 degree domain (-pi/2, pi/2], atan2 returns an angle defined on a 360 degree domain (-pi, pi].

Derivatives

- A derivative measures how much a function changes as its input changes.
- For a function $y(t)$, the derivative (with respect to time) measures how much y changes when t changes.
- The derivative provides the slope of a curve at a given point. It is aligned to the *tangent* of the curve.



Derivatives

- The process of finding a derivative is called *differentiation*
- There are some basic rules:
  - The derivative of a constant (denoted with *c* below) is 0. This is intuitive because a constant value does not change.

$$\frac{d}{dt}c = 0$$

  - The *power rule* provides derivatives for integer powers of *t*

$$\frac{d}{dt}t^n = nt^{n-1}$$

What is the derivative of x(t) = 5?

$$\frac{d}{dt}5 = 0$$

What is the derivative of x(t) = t^2?

$$\frac{d}{dt}t^2 = 2t$$

Derivatives

- *Constant multiple rule*: If x is a differentiable function of t, and c is a constant, then

$$\frac{d}{dt}(cx) = c\frac{dx}{dt}$$

- *Sum rule:* The derivative of a sum is the sum of derivatives

$$\frac{d}{dt}(x_1 + x_2) = \frac{dx_1}{dt} + \frac{dx_2}{dt}$$

What is the derivative of $x(t) = 5t^2 + 5t + 5$?

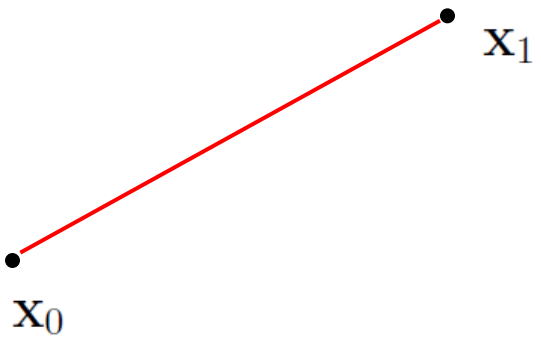
$$\begin{aligned}
\frac{d}{dt}(5t^2 + t5 + 5) &= \frac{d}{dt}(5t^2) + \frac{d}{dt}(5t) + \frac{d}{dt}(5) \\
&= 5\frac{d}{dt}(t^2) + 5\frac{d}{dt}(t) + \frac{d}{dt}(5) \\
&= 5(2t) + 5(1) + 0 \\
&= 10t + 5
\end{aligned}$$

Space curve

- A space curve is simply a curve in space. There are different ways to model a space curve. A *parametric* form represents *x*, *y*, and *z* as functions of *t*:

$$\mathbf{x}(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix}$$

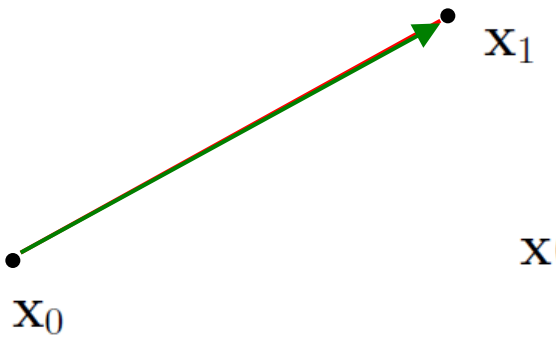
- As a simple example, consider the line segment between points $\mathbf{x}_0$ and $\mathbf{x}_1$.



$$\mathbf{x}(t) = (1-t)\mathbf{x}_0 + t\mathbf{x}_1$$

Derivative of a space curve

- A derivative of a space curve can be taken as well, following the same rules as before (only now involving vectors).
- What is ? $\frac{d}{dt}\mathbf{x}(t)$



$$\mathbf{x}(t) = (1-t)\mathbf{x}_0 + t\mathbf{x}_1$$

- The derivative is $\mathbf{x}_1 - \mathbf{x}_0$.
- The derivative of a space curve provides a vector *tangent* to the curve. It provides a vector showing which way the curve is going.

Exercise

1. Derive a mathematical expression for the derivative (tangent) of the following cubic space curve, where **a**, **b**, **c**, and **d** are constants.

$$\mathbf{x}(t) = \mathbf{a} + \mathbf{b}t + \mathbf{c}t^2 + \mathbf{d}t^3$$

2. What is the derivative evaluated at *t* = 0?
3. What is the derivative evaluated at *t* = 1?

- Hint: remember the power rule.

$$\frac{d}{dt}t^n = nt^{n-1}$$

- Answer:

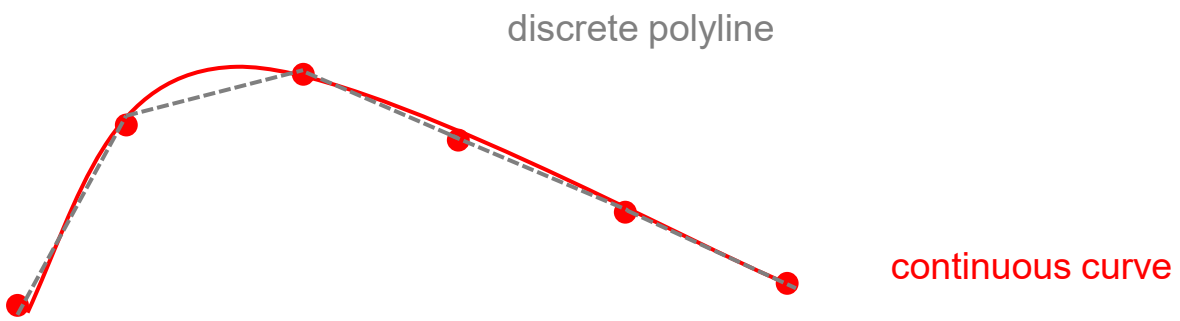
$$\frac{d\mathbf{x}(t)}{dt} = \mathbf{b} + 2\mathbf{c}t + 3\mathbf{d}t^2$$

$$\left.\frac{d\mathbf{x}(t)}{dt}\right|_{t=0} = \mathbf{b}$$

$$\left.\frac{d\mathbf{x}(t)}{dt}\right|_{t=1} = \mathbf{b} + 2\mathbf{c} + 3\mathbf{d}$$

Splines

- Splines provide a continuous mathematical description of a *space curve*.
- The spline can be sampled to produce a discrete polyline for rendering.
- There are many types of splines.
  - Linear spline: has the form $\mathbf{a} + \mathbf{b}t$, where t is time and **a** and **b** are constants
  - Quadratic spline: $\mathbf{a} + \mathbf{b}t + \mathbf{c}t^2$ (can wiggle)
  - Cubic spline: $\mathbf{a} + \mathbf{b}t + \mathbf{c}t^2 + \mathbf{d}t^3$ (can wiggle more)
- Any *continuous* mathematical geometry must be *sampled* to produce a discrete representation before rendering



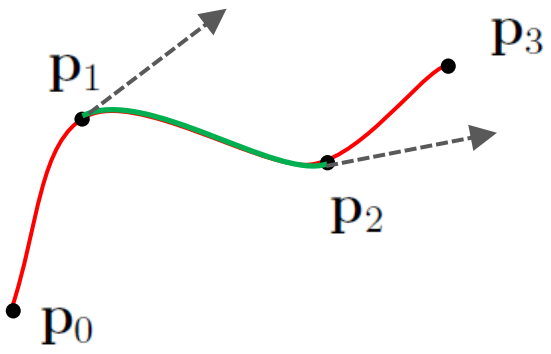
Catmull-Rom splines

- Catmull-Rom splines interpolate a set of **control points**, and are guaranteed to *pass through* the control points (except for the first and last points).
- A Catmull-Rom spline has the following properties:
  - It passes through the middle two points, and interpolates the space between them

$$\mathbf{x}(t=0) = \mathbf{p}_1$$
$$\mathbf{x}(t=1) = \mathbf{p}_2$$

  - The tangents at $\mathbf{p}_1$ and $\mathbf{p}_2$ are

$$\left.\frac{d\mathbf{x}}{dt}\right|_{t=0} = \frac{\mathbf{p}_2 - \mathbf{p}_0}{2}$$
$$\left.\frac{d\mathbf{x}}{dt}\right|_{t=1} = \frac{\mathbf{p}_3 - \mathbf{p}_1}{2}$$



- Since there are *four constraints*, we will have *four unknowns* in the spline equation (i.e., we will have a cubic spline, $\mathbf{a} + \mathbf{b}t + \mathbf{c}t^2 + \mathbf{d}t^3$)

Catmull-Rom splines

- Cubic spline equation $\mathbf{x}(t) = \mathbf{a} + \mathbf{b}t + \mathbf{c}t^2 + \mathbf{d}t^3$
- The goal is to derive $\mathbf{a}, \mathbf{b}, \mathbf{c}$, and $\mathbf{d}$ using the constraints. This will then give the precise form of **x**(*t*).
- The first constraint is that $\mathbf{x}(t=0) = \mathbf{p}_1$. What is $\mathbf{x}(t=0)$?

$$\mathbf{x}(t=0) = \mathbf{a} = \mathbf{p}_1$$

- The second constraint is that $\mathbf{x}(t=1) = \mathbf{p}_2$. What is $\mathbf{x}(t=1)$?

$$\mathbf{x}(t=1) = \mathbf{a} + \mathbf{b} + \mathbf{c} + \mathbf{d} = \mathbf{p}_2$$

- The third constraint is that $\left.\frac{d\mathbf{x}}{dt}\right|_{t=0} = \frac{\mathbf{p}_2 - \mathbf{p}_0}{2}$. What is $\left.\frac{d\mathbf{x}}{dt}\right|_{t=0}$?

$$\left.\frac{d\mathbf{x}}{dt}\right|_{t=0} = \mathbf{b} = \frac{\mathbf{p}_2 - \mathbf{p}_0}{2}$$

- The fourth constraint is that $\left.\frac{d\mathbf{x}}{dt}\right|_{t=1} = \frac{\mathbf{p}_3 - \mathbf{p}_1}{2}$. What is $\left.\frac{d\mathbf{x}}{dt}\right|_{t=1}$?

$$\left.\frac{d\mathbf{x}}{dt}\right|_{t=1} = \mathbf{b} + 2\mathbf{c} + 3\mathbf{d} = \frac{\mathbf{p}_3 - \mathbf{p}_1}{2}$$

Catmull-Rom splines

- Four linear equations with four unknowns.

$$\mathbf{a} = \mathbf{p}_1$$

$$\mathbf{a} + \mathbf{b} + \mathbf{c} + \mathbf{d} = \mathbf{p}_2$$

$$\mathbf{b} = \frac{\mathbf{p}_2 - \mathbf{p}_0}{2}$$

$$\mathbf{b} + 2\mathbf{c} + 3\mathbf{d} = \frac{\mathbf{p}_3 - \mathbf{p}_1}{2}$$

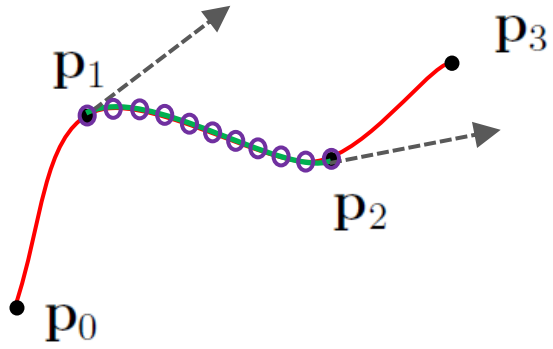
- This is easily solved using algebra to determine **a**, **b**, **c**, and **d**!

Catmull-Rom splines

- The four equations on the previous slide can be solved, giving the result

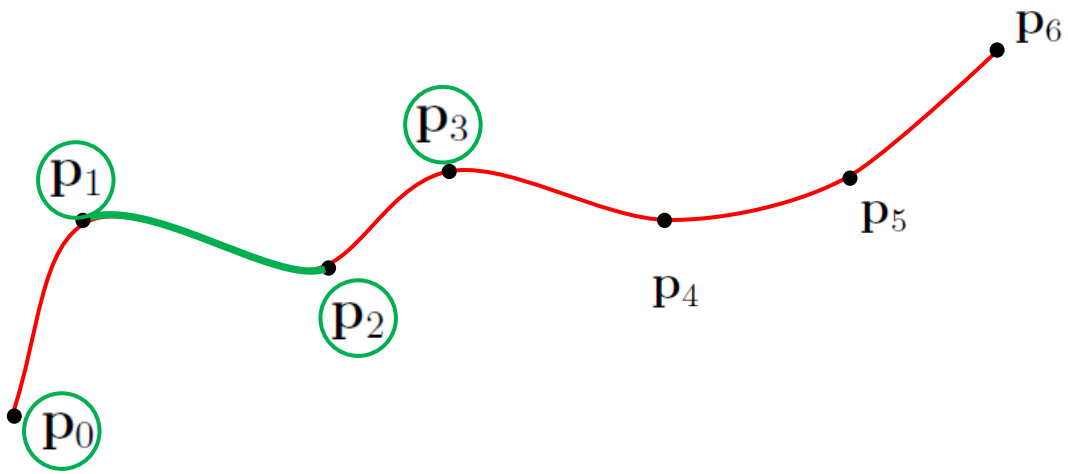
$$
\begin{aligned}
\mathbf{a} &= \mathbf{p}_1 \\
\mathbf{b} &= \frac{1}{2}(-\mathbf{p}_0 + \mathbf{p}_2) \\
\mathbf{c} &= \frac{1}{2}(2\mathbf{p}_0 - 5\mathbf{p}_1 + 4\mathbf{p}_2 - \mathbf{p}_3) \\
\mathbf{d} &= \frac{1}{2}(-\mathbf{p}_0 + 3\mathbf{p}_1 - 3\mathbf{p}_2 + \mathbf{p}_3)
\end{aligned}
$$

⇨ Given four control points ($\mathbf{p}_0$, $\mathbf{p}_1$, $\mathbf{p}_2$, $\mathbf{p}_3$), we can compute **a**, **b**, **c**, and **d**. Then, we can sample the curve $\mathbf{x}(t) = \mathbf{a} + \mathbf{b}t + \mathbf{c}t^2 + \mathbf{d}t^3$ at regular intervals along the interval t=[0, 1] to generate points along the spline spanning the space between $\mathbf{p}_1$ and $\mathbf{p}_2$.



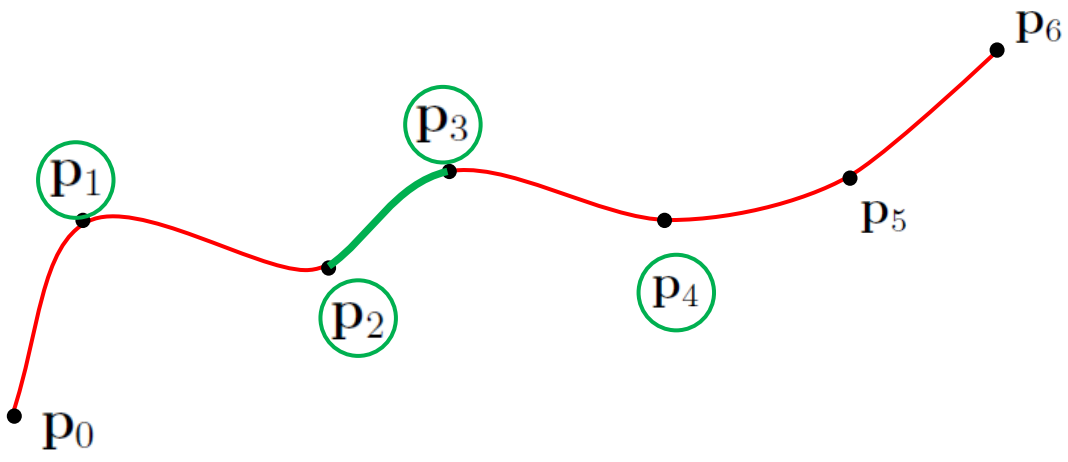
Catmull-Rom splines

- Question: The equation on the previous slide describes a spline between two control points. What if we have many control points?
- Answer: Group each four consecutive points to form a spline segment.



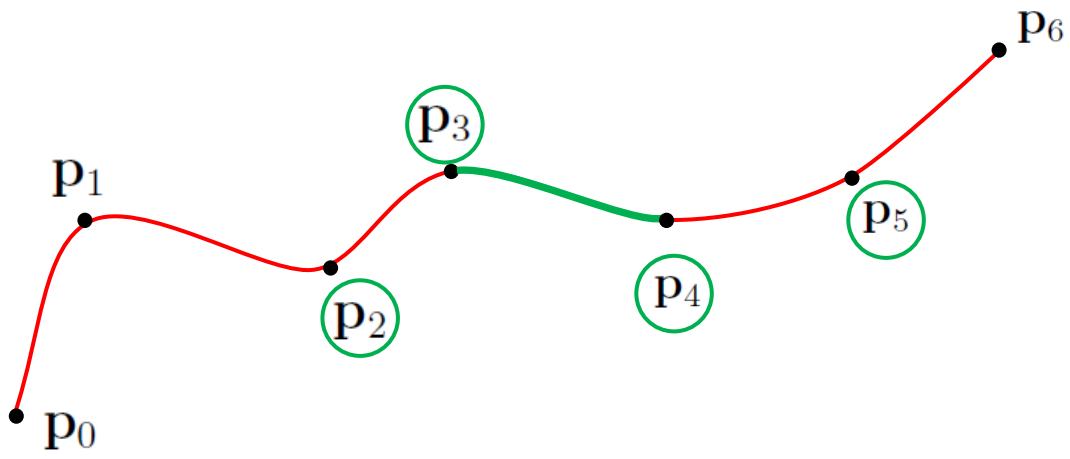
Catmull-Rom splines

- Question: The equation on the previous slide describes a spline between two control points. What if we have many control points?
- Answer: Group each four consecutive points to form a spline segment.



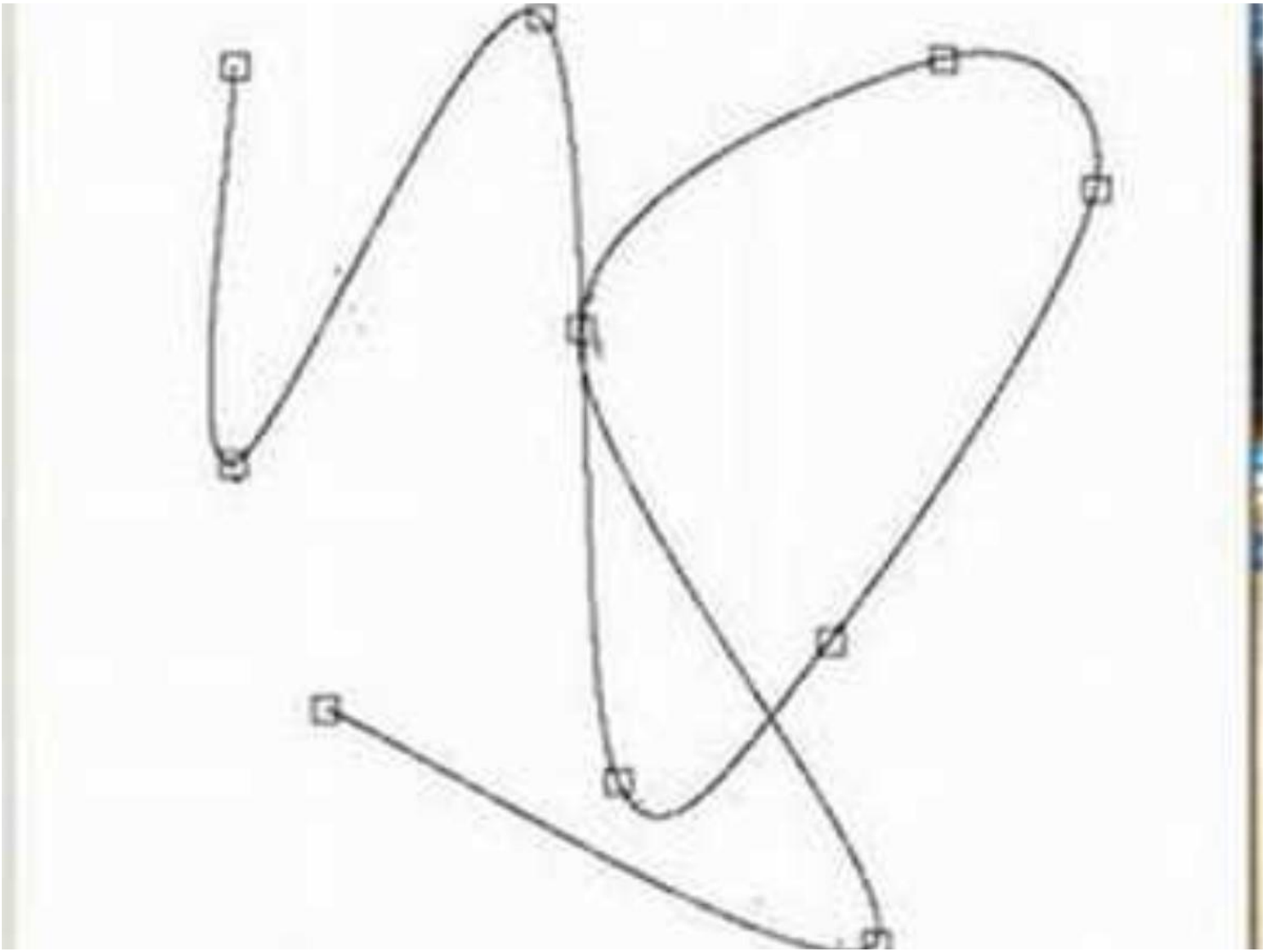
Catmull-Rom splines

- Question: The equation on the previous slide describes a spline between two control points. What if we have many control points?
- Answer: Group each four consecutive points to form a spline segment.



- If there are *N* control points, we can define *N – 3* spline segments (open curve).
- Splines defined this way have C^1 continuity, meaning they are continuous and have continuous derivatives.
- Nice video: https://www.youtube.com/watch?v=0BDJhfbLOY8

Catmull-Rom Applet



Coursework

- A great opportunity to develop an exciting application that incorporates taught content whilst creating an element for your portfolio!

⇨ **It's time to start**

- Suggestions:
  - Look at the Vector and Matrix worksheets to understand the bits about paths
  - Do lab 3 to get familiar with using the camera and VBOs
  - Start by modelling the centreline of your path using a Catmull-Rom spline
  - Then try moving the camera along the spline path
  - Once you have this, try building the path geometry and place into a VBO for rendering.

- The "Path Starting Guide" on Moodle will help guide you on the basics of these steps.

- Interim coursework description will also be presented at the lecture